Julius Bär

BREAK THE SILO

How we built a decentralised control plane for service orchestration

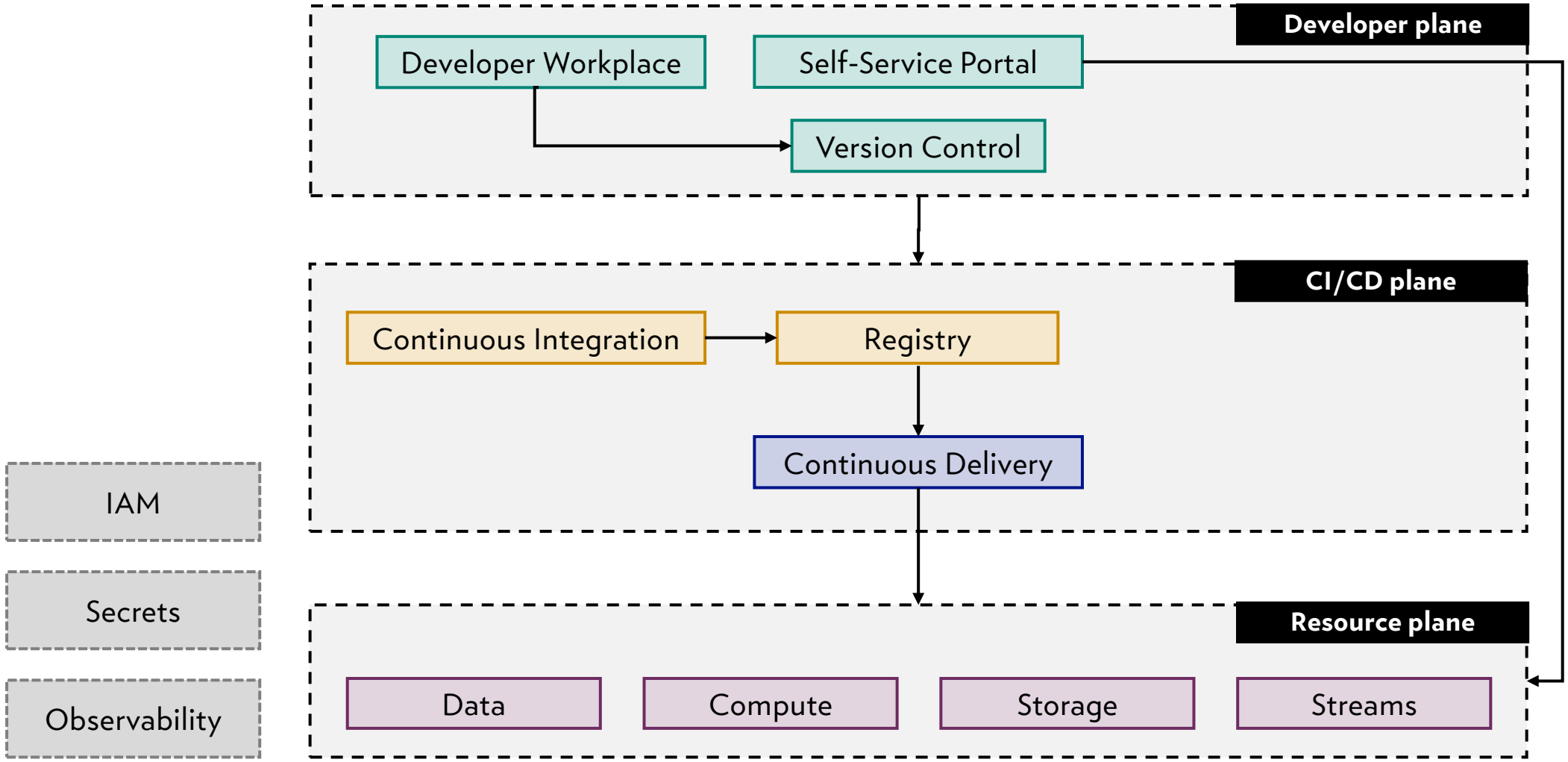
Voxxed Days Ticino 2026

By Eros Taborelli – Principal Engineer & Software Architect at Julius Baer

WHY?

Context

Internal DevOps Platform



Where we come from

The ‘traditional’ way of ordering infrastructure services

An engineer

Makes an order on a portal

And receives a resource (example: VM)

Why it works

Benefits of a UI

- Filling a form is a normal, mundane task
- Immediate validation and visual feedback
- Some products and services come with a portal by default

Shortcomings

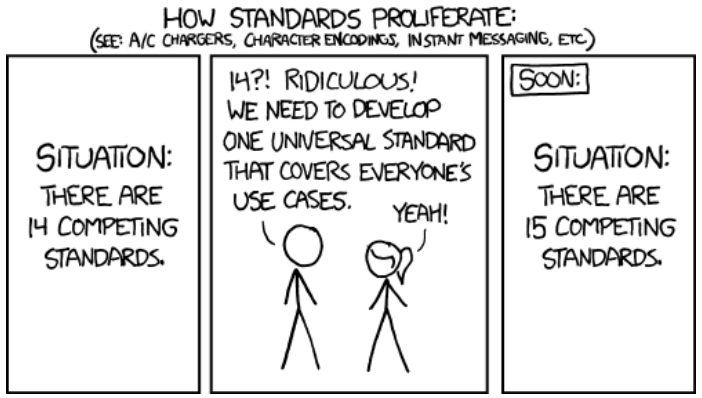
Hidden costs of a UI

- Day 2 operations must be implemented
- No comparison between environments
- Repeatability/Replicability

Split Responsibilities

Multiple teams involved, unclear ownership for the full process

ONE PORTAL



Portal Team

Automation Team

Infrastructure Team

Infrastructure Team

Infrastructure Team

Product Portal

Automation Engine

Infrastructure Product

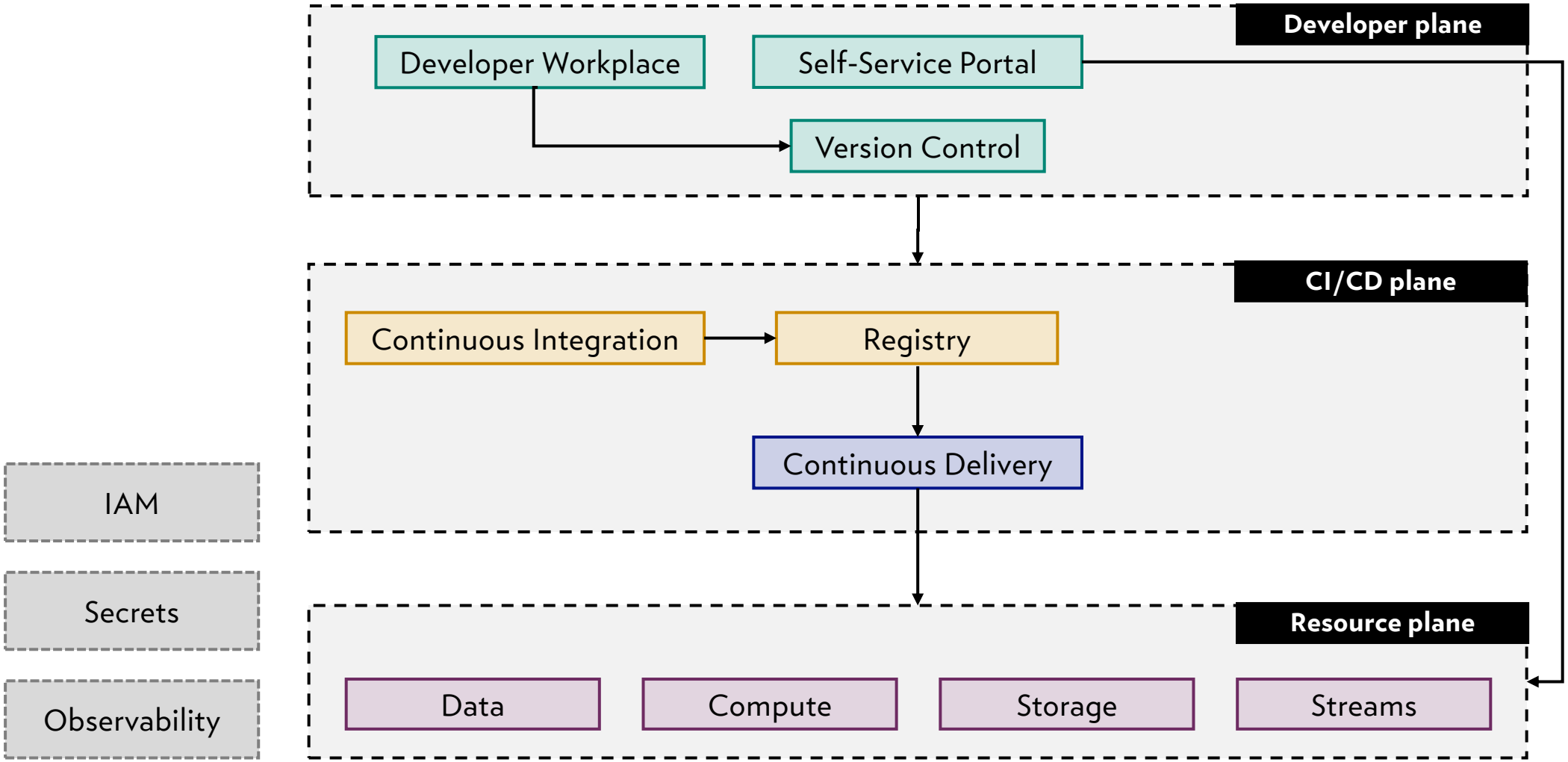
Product Portal

Automation Engine

Infrastructure Product

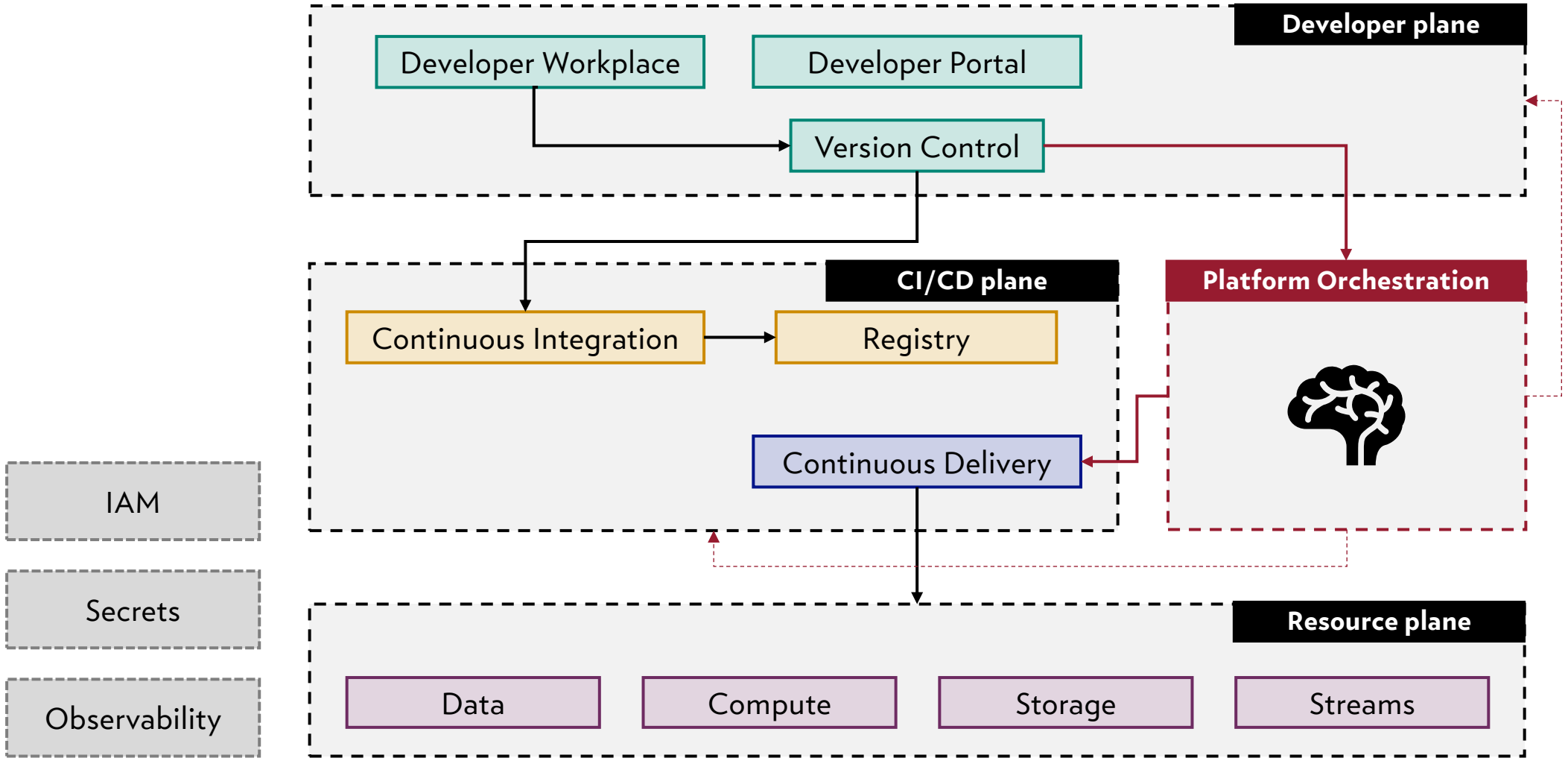
Context

Internal DevOps Platform



Context

Internal DevOps Platform



Our goals

Standardisation

Consistent experience when consuming infrastructure

Operational excellence

Actively promote the usage of Infra-as-Code

Ownership

Infrastructure Teams to drive the offering to end customers

WHY?

Our goals

Standardisation

Consistent experience when consuming infrastructure

- Familiar APIs with common patterns: Service Catalog
- No key differences when working with a database, a Kafka topic, or an object storage

Operational excellence

Actively promote the usage of Infra-as-Code

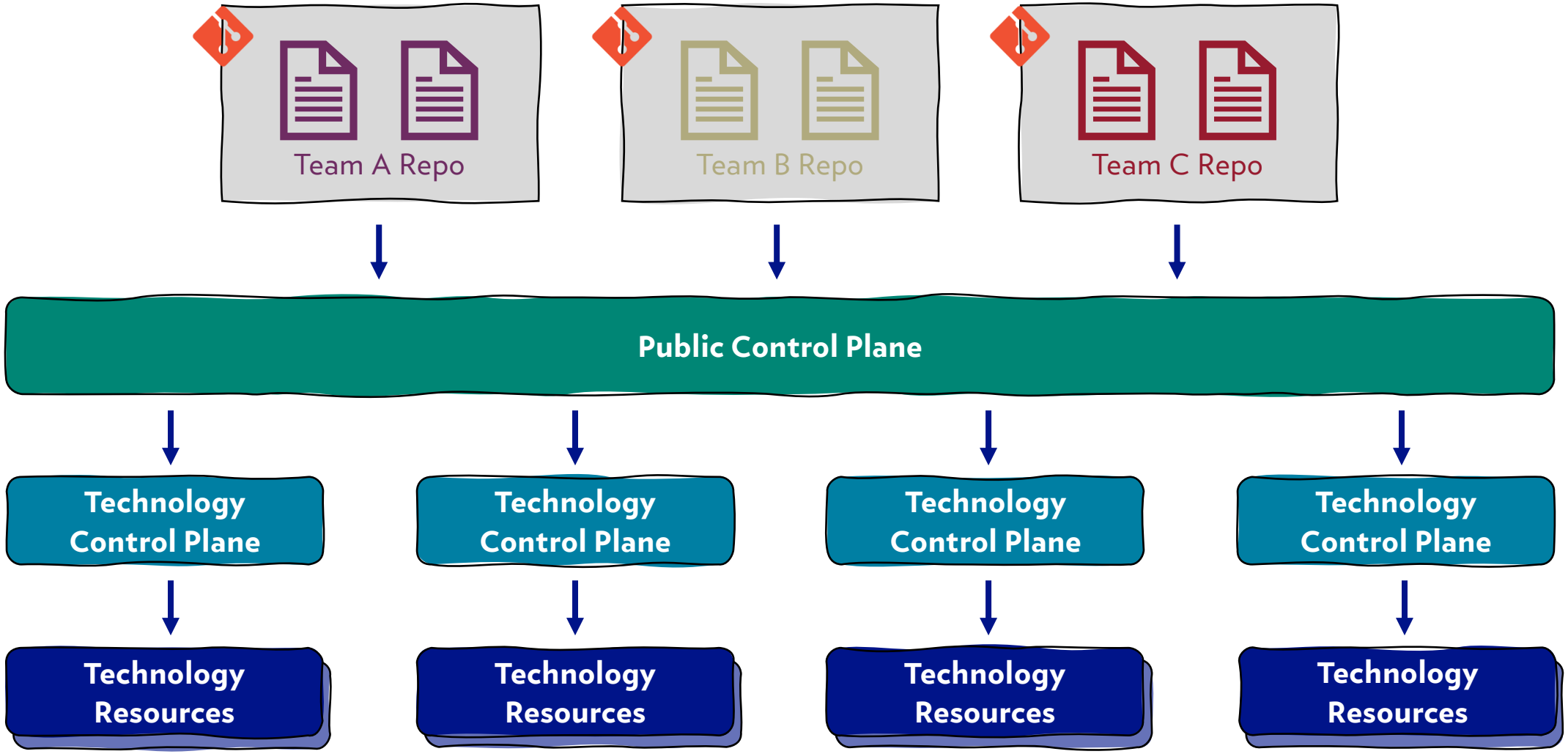
- Goodbye ClickOps
- Repeatable Infrastructure
- Built-In audit, diff, and day2 operations
- Apply Software Development best practices to infrastructure

Ownership

Infrastructure Teams to drive the offering to end customers

- Reduce dependencies
- End-to-End experience owned by the infrastructure teams, regardless of organizational structures
- More direct customer feedback

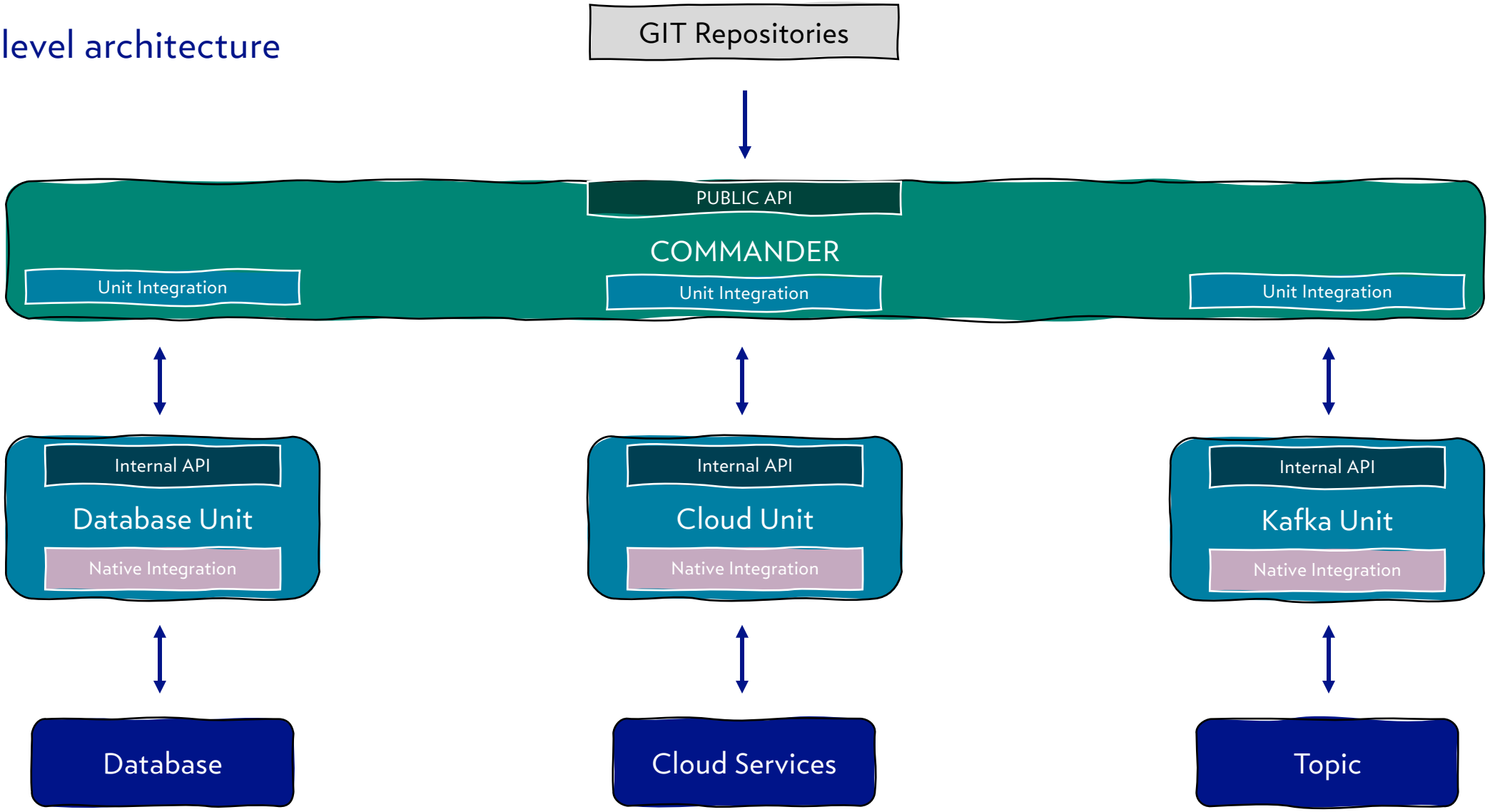
Concept


Team A Repo
Team B Repo
Team C Repo
Public Control Plane
Technology Control Plane
Technology Control Plane
Technology Control Plane
Technology Control Plane
Technology Resources
Technology Resources
Technology Resources
Technology Resources

WELCOME: FLEET COMMANDER

Fleet commander

High level architecture



Goals achieved?

- ✓ Operational Excellence: GitOps / X-as-Code
- ✓ Standardization: Service Catalog
- ? Ownership: Decentralized Architecture and Responsibilities

KUBERNETES

+

CROSSPLANE

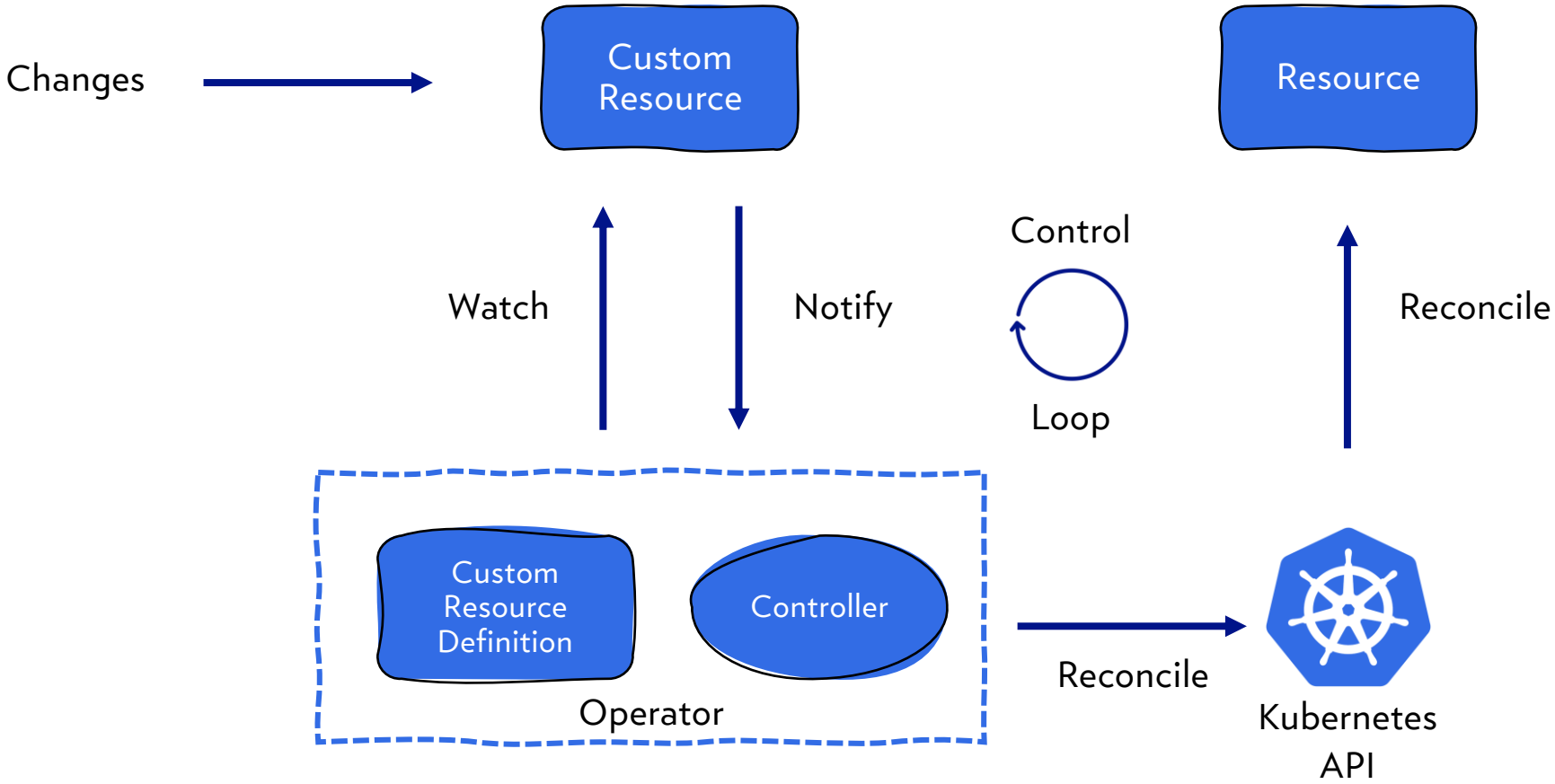
=

FLEET COMMANDER

WHY?

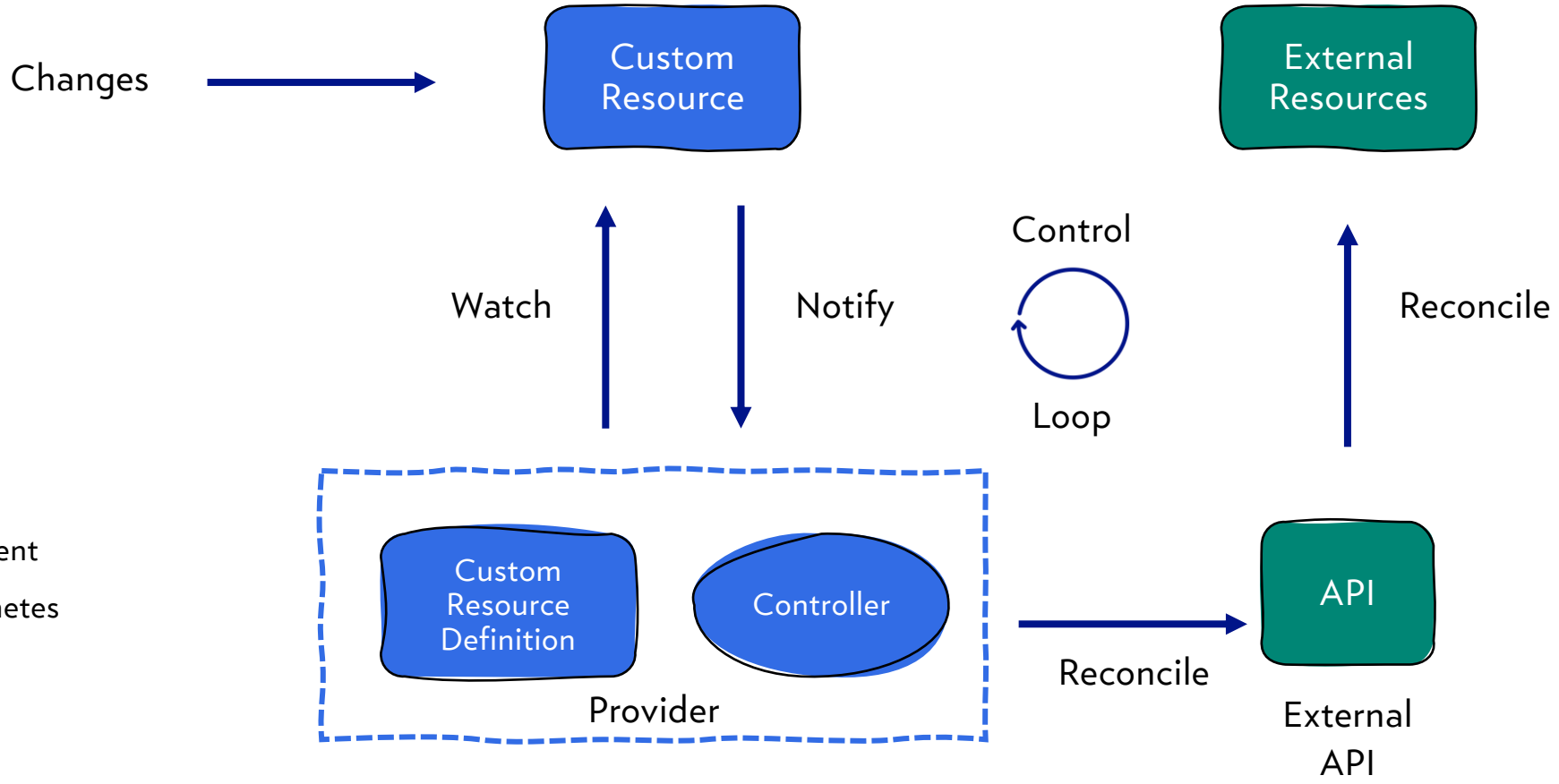
Operator pattern

Control loop on custom resources



Crossplane providers

Operators for external resources



Examples

- provider-azure
- provider-aws
- provider-gcp
- provider-confluent
- provider-kubernetes
- provider-vault

Crossplane composition

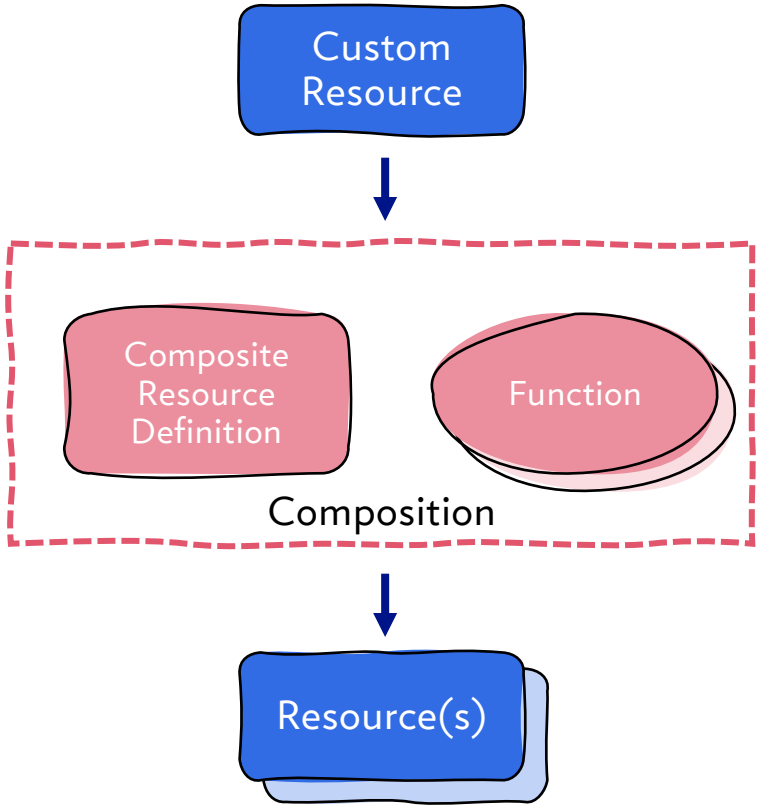
Harness the power of custom resource definitions

Composite resource definition

- Define the **Public API** of the Composition.
- When the Composition is deployed, Crossplane creates a standard Kubernetes **Custom Resource Definition**.

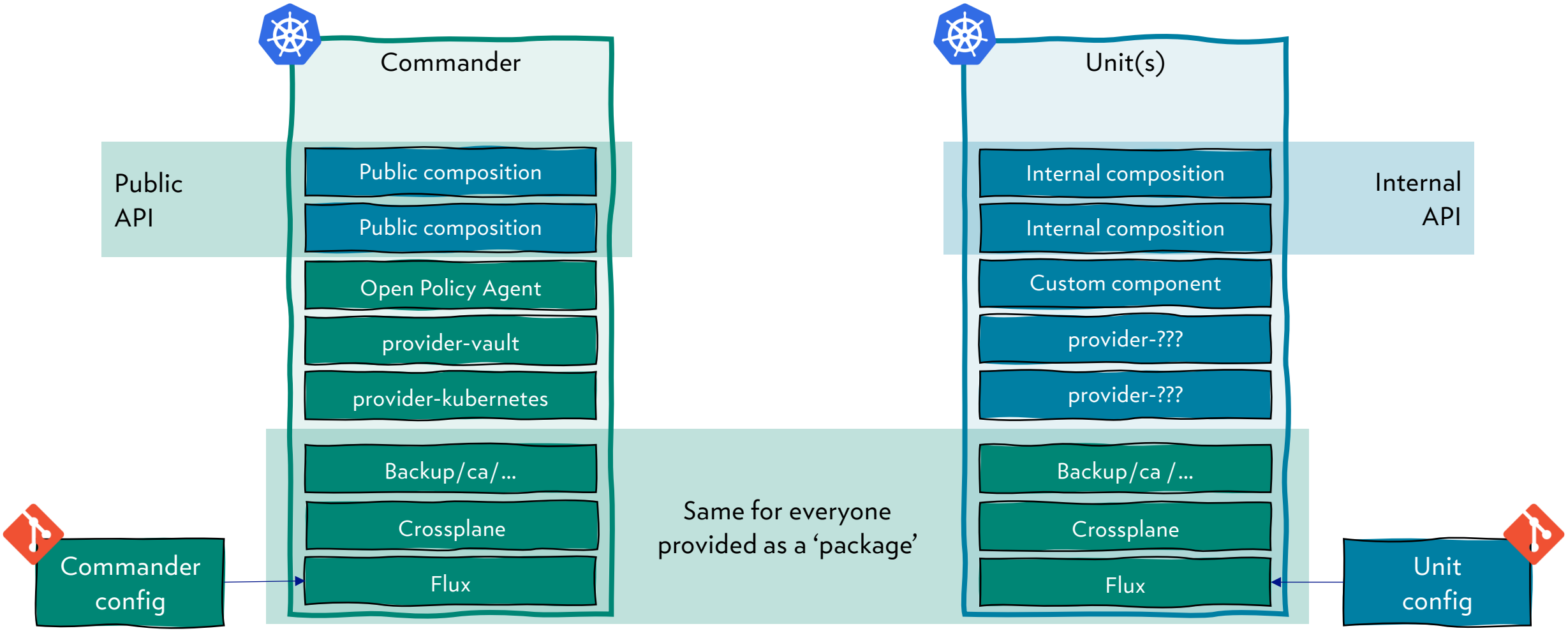
Function

- The implementation of the composition.
- Handles the CR and **decompose to one or multiple Kubernetes resources.**
- (hint: including resources defined by Crossplane providers and other compositions)



Fleet commander

Shared responsibility model

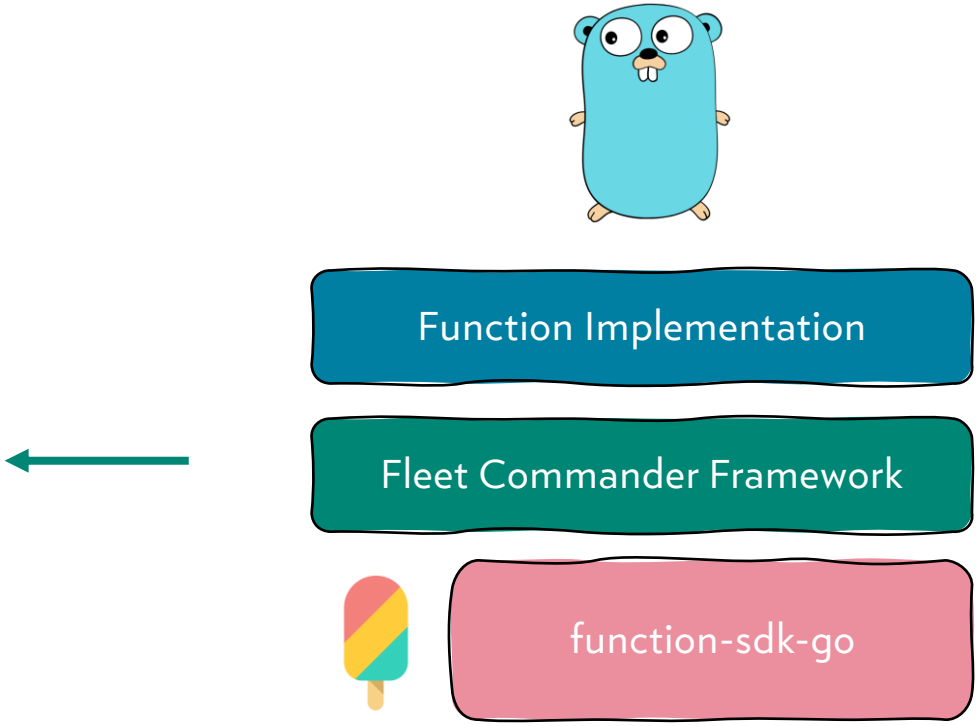


SDK

What fleet commander offers for easier onboarding

Fleet commander framework

- Golang: type safety, XRD generation, sustainable long-term
- Utility functions
  - provider-kubernetes wrapper (more later)
  - labels & annotations
  - unit testing
- Generic Helm chart (unit configuration)
- Build system



Goals achieved?

- ✓ Operational Excellence: GitOps / X-as-Code
- ✓ Standardization: Service Catalog
- ✓ Ownership: Decentralized Architecture and Responsibilities

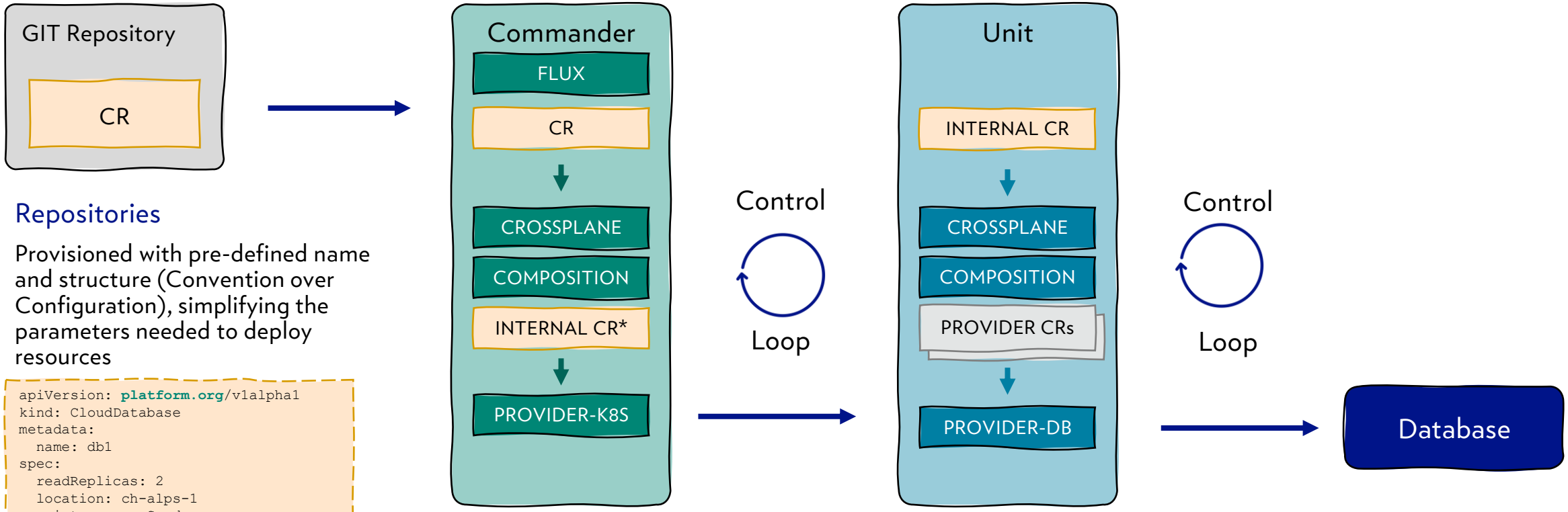
BUT HOW DOES IT WORK?

Example

Provision a generic resource

*: wrapped for the k8s provider

```yaml
apiVersion: kubernetes.crossplane.io/v1alpha2
kind: Object
spec:
  forProvider:
    apiVersion: internal.platform.org/v1alpha1
    kind: CloudDatabase
    spec:
      readReplicas: 2
      location: ch-alps-1
      maintenance: Sunday
  providerConfigRef:
    name: dbunit
```



Repositories

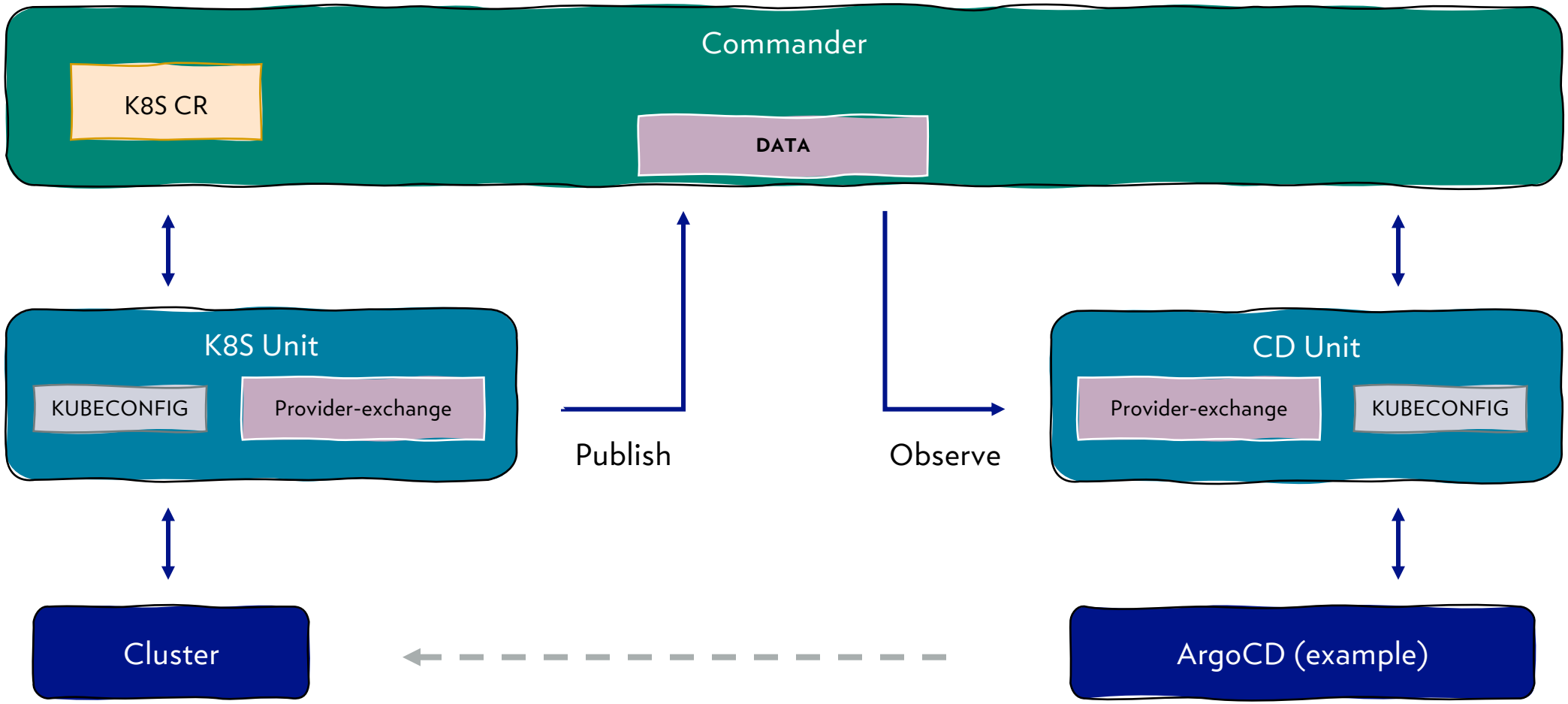
Provisioned with pre-defined name and structure (Convention over Configuration), simplifying the parameters needed to deploy resources

```yaml
apiVersion: platform.org/v1alpha1
kind: CloudDatabase
metadata:
  name: db1
spec:
  readReplicas: 2
  location: ch-alps-1
  maintenance: Sunday
```

CHALLENGES

Data exchange

Solve dependencies between units (example)



Lifecycle

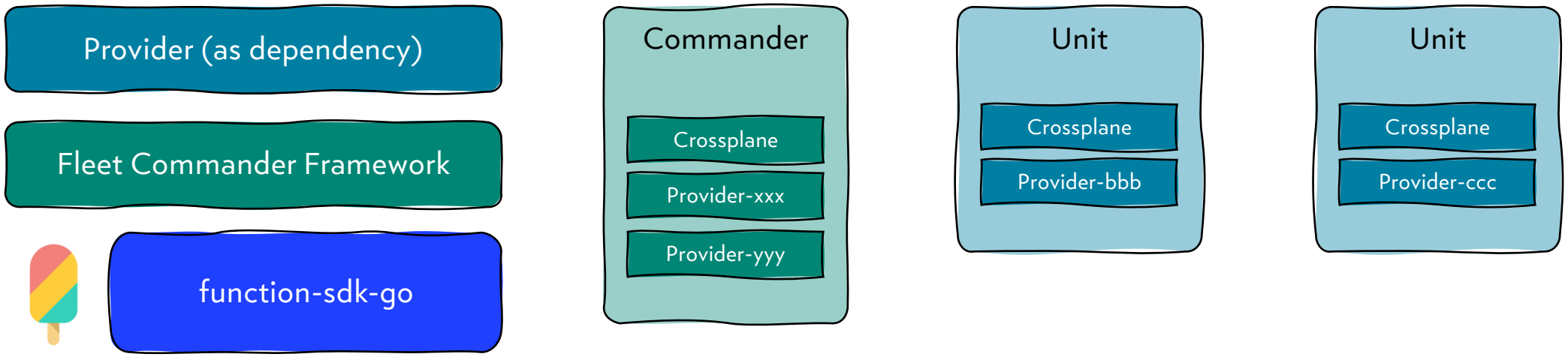
How to handle Crossplane updates in a distributed environment

Crossplane versions

- Crossplane runtime compatibility
- Crossplane libraries compatibility
- Fleet Commander Framework compatibility

Providers' requirements

- Providers have different Crossplane version requirements
- Different providers may have conflicting requirements



Onboarding costs

Steep learning curve

Learn Kubernetes concepts

- CRDs
- Operators

Learn Crossplane

- Providers
- Compositions
- Functions

Get familiar with Fleet Commander

- Architecture
- Framework
- Golang

Operate Unit

- Kubernetes
- Dependencies
- Maintenance

Q&A

THANK YOU!

github.com/juliusbaer/conferences

eros.taborelli@juliusbaer.com

Fleet Commander building blocks

Crossplane

Main project and releases

https://github.com/crossplane/crossplane

Develop compositions using Go

https://github.com/crossplane/function-sdk-go

Kubernetes Provider

Deploy and Manage arbitrary Kubernetes objects on different clusters (commander to unit)https://github.com/crossplane-contrib/provider-kubernetes

two Kubernetes

commander

- crossplane
- provider-kubernetes
  - configure unit credentials

- write public composition
  - decomposes into a kubernetes object
  - kubernetes object wraps internal composition custom resource
  - reference the unit configuration to delegate

- apply custom resource

unit

- crossplane
- desired providers

- write internal composition
  - decomposes into provider custom resources

- it appears here